

Section A

1. Why is Node.js preferred for real-time backend APIs?

Node.js is widely used for real-time applications because it uses an event-driven, non-blocking architecture. Instead of creating a new thread for every request, Node.js handles multiple client requests efficiently using a single-threaded event loop.

Advantages:

- High performance for I/O-intensive applications
- Handles thousands of concurrent connections
- Ideal for chat applications, live notifications, gaming servers, and streaming services
- Uses JavaScript on both frontend and backend

Example Use Cases:

- WhatsApp Web
 - Real-time chat applications
 - Live stock market updates
 - Online multiplayer games
-

2. Explain Non-Blocking I/O Using a File Read Example

In traditional blocking I/O, the program waits until the file is completely read before executing the next statement.

Blocking Example

```
const fs = require('fs');

const data = fs.readFileSync('sample.txt', 'utf8');
console.log(data);

console.log("Program Finished");
```

Execution Flow:

1. Read file
2. Wait until reading completes
3. Print file contents
4. Print "Program Finished"

Non-Blocking Example

```
const fs = require('fs');

fs.readFile('sample.txt', 'utf8', (err, data) => {
  console.log(data);
});

console.log("Program Finished");
```

Execution Flow:

1. Start reading file
2. Continue executing next statements
3. Print "Program Finished"
4. Display file contents when reading completes

Output:

Program Finished
File Content Here...

This allows Node.js to serve other requests while waiting for file operations.

3. How Promises Prevent Callback Hell

Callback Hell

```
getUser(function(user){
  getOrders(user.id, function(orders){
    getPayment(orders[0].id, function(payment){
      console.log(payment);
    });
  });
});
```

Problems:

- Deep nesting
 - Difficult debugging
 - Poor readability
 - Hard maintenance
-

Using Promises

```
getUser()
  .then(user => getOrders(user.id))
  .then(orders => getPayment(orders[0].id))
  .then(payment => console.log(payment))
  .catch(error => console.log(error));
```

Benefits:

- Cleaner code
 - Better error handling using .catch()
 - Easy chaining of asynchronous operations
 - Improved readability
-

Using Async/Await

```
async function fetchData() {
  try {
    const user = await getUser();
    const orders = await getOrders(user.id);
    const payment = await getPayment(orders[0].id);

    console.log(payment);
  } catch(error) {
    console.log(error);
  }
}
```

This is the most readable approach for handling asynchronous operations.

4. Explain MongoDB Document Structure for Product Catalogs

MongoDB stores data in documents using BSON (Binary JSON) format.

Product Document Example

```
{
  "_id": "101",
  "name": "Laptop",
  "category": "Electronics",
  "price": 55000,
  "stock": 20,
  "brand": "Dell",
```

```
"features": [  
  "16GB RAM",  
  "512GB SSD",  
  "Intel i7"  
]  
}
```

Field Types

Field	BSON Type
_id	ObjectId/String
name	String
category	String
price	Number
stock	Number
brand	String
features	Array

Advantages

- Flexible schema
- Easy storage of nested data
- No need for complex table joins
- Suitable for e-commerce product catalogs

Example with nested specifications:

```
{  
  "name": "Smartphone",  
  "price": 30000,  
  "specifications": {  
    "ram": "8GB",  
    "storage": "128GB",  
    "camera": "50MP"  
  }  
}
```

5. Why Are Aggregation Pipelines Essential for Reports?

The MongoDB Aggregation Framework processes data through multiple stages to generate analytics and reports.

Example Collection

```
{
  "name": "Laptop",
  "category": "Electronics",
  "price": 50000,
  "stock": 10
}

{
  "name": "Mouse",
  "category": "Electronics",
  "price": 1000,
  "stock": 20
}
```

Aggregation Query

```
db.products.aggregate([
{
  $group: {
    _id: "$category",
    totalStock: { $sum: "$stock" },
    totalValue: {
      $sum: { $multiply: ["$price", "$stock"] }
    }
  }
}
])
```

Result

```
{
  "_id": "Electronics",
  "totalStock": 30,
  "totalValue": 520000
}
```

Why Aggregation Pipelines Are Important

1. Generate Business Reports
 - Sales reports
 - Inventory reports

- Revenue analysis
- 2. Perform Data Summarization
 - Total stock by category
 - Average product price
 - Monthly sales totals
- 3. Improve Performance
 - Computations occur inside MongoDB
 - Reduces application-side processing
- 4. Support Real-Time Analytics
 - Dashboards
 - Inventory monitoring
 - Product performance tracking

Common stages include:

- \$match → Filter data
- \$group → Group records
- \$sum → Calculate totals
- \$avg → Calculate averages
- \$sort → Order results
- \$project → Select required fields

Aggregation pipelines are essential because they transform raw MongoDB data into meaningful business insights efficiently and in real time.